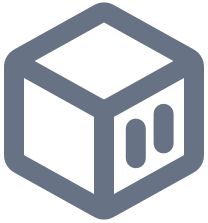


[Explore](#) / [gvenzl](#) / oracle-xe

gvenzl/oracle-xe

By [gvenzl](#) • Updated almost 2 years ago

Oracle Database XE (21c, 18c, 11g) for everyone! See [gvenzl/oracle-free](#) for Oracle Database 23c Free

IMAGE

DATABASES & STORAGE

☆ 364 ↓ 10M+

Overview

Tags

oci-oracle-xe

Oracle Database Express Edition Container / Docker images.

The images are compatible with `podman` and `docker`. You can use `podman` or `docker` interchangeably.

Supported tags and respective `Dockerfile` links

- `latest` , `21` , `21.3.0` [↗](#), `latest-faststart` , `21-faststart` , `21.3.0-faststart` [↗](#)

Tag summary

Recent tags

latest



Content type

Image

Digest

sha256:c2682a421... [↗](#)

Size

- [slim](#) , [21-slim](#) , [21.3.0-slim](#) [↗](#), [slim-faststart](#) , [21-slim-faststart](#) , [21.3.0-slim-faststart](#) [↗](#)
- [full](#) , [21-full](#) , [21.3.0-full](#) [↗](#), [full-faststart](#) , [21-full-faststart](#) , [21.3.0-full-faststart](#) [↗](#)
- [18](#) , [18.4.0](#) [↗](#), [18-faststart](#) , [18.4.0-faststart](#) [↗](#)
- [18-slim](#) , [18.4.0-slim](#) [↗](#), [18-slim faststart](#) , [18.4.0-slim-faststart](#) [↗](#)
- [18-full](#) , [18.4.0-full](#) [↗](#), [18-full-faststart](#) , [18.4.0-full-faststart](#) [↗](#)
- [11](#) , [11.2.0.2](#) [↗](#), [11-faststart](#) , [11.2.0.2-faststart](#) [↗](#)
- [11-slim](#) , [11.2.0.2-slim](#) [↗](#), [11-slim-faststart](#) , [11.2.0.2-slim-faststart](#) [↗](#)
- [11-full](#) , [11.2.0.2-full](#) [↗](#), [11-full-faststart](#) , [11.2.0.2-full-faststart](#) [↗](#)

Quick Start

Run a new database container (data is removed when the container is removed, but kept throughout container restarts):

```
docker run -d -p 1521:1521 -e ORACLE_PASSWORD=<your password> gvenzl/oracle-xe
```

Run a new persistent database container (data is kept throughout container lifecycles):

```
docker run -d -p 1521:1521 -e ORACLE_PASSWORD=<your password> -v oracle-volume:/opt/oracle/oradata gvenzl/oracle-xe
```

Run a new persistent **11g R2** database container (volume path differs in 11g R2):

```
docker run -d -p 1521:1521 -e ORACLE_PASSWORD=<your password> -v oracle-volume:/u01/app/oracle/oradata gvenzl/oracle-xe:11
```

1.4 GB

Last updated

almost 2 years ago

```
docker pull gvenzl/oracle-xe
```



Reset database `SYS` and `SYSTEM` passwords:

```
docker exec <container name|id> resetPassword <your password>
```

Oracle XE on Apple M chips

Currently, there is no Oracle Database port for ARM chips, hence Oracle XE images cannot run on the new Apple M chips via Docker Desktop.

Fortunately, there are other technologies that can spin up `x86_64` software on Apple M chips, such as [colima](#). To run these Oracle XE images on Apple M hardware, follow these simple steps:

- Install colima ([instructions](#))
- Run `colima start --arch x86_64 --memory 4`
- Start container as usual

Users of these images

We are proud of the following users of these images:

- [Airbyte](#) [`6e53a57`]
- [Apache Spark](#) [`e03afc9`]
- [Benthos](#) [`c2a8b6a`] (migrated to [gvenzl/oracle-free](#))
- [Ebean](#) [`ae8f1cd`]
- [Eclipse Vert.x](#) [`ffedbce`]
- [Flowable](#) [`18c751f`]
- [GeoTools](#) [`f922f0b`]
- [Hibernate](#) [`43d2274`]
 - [Hibernate Reactive](#) [`7de7d79`] (migrated to [gvenzl/oracle-free](#))
 - [Hibernate Search](#) [`173f0b7`]
- [jOOQ](#) [`#35`] (migrated to [gvenzl/oracle-free](#))

- [Liquibase](#) [`c6a31c0`]
- [Micronaut Data](#) [`ddf11c1`]
- [Quarkus](#) [`9a63a58`] (migrated to [gvenzl/oracle-free](#))
- [Ruby API for Oracle PL/SQL](#) [`63baad0`]
- [Ruby on Rails ActiveRecord adapter](#) [`afd7a93`] (migrated to [gvenzl/oracle-free](#))
- [Rucio by CERN](#) [`80dffbb`]
- [SchemaCrawler](#) [`08d9b87`]
- [Spring Data JDBC](#) [`bace76a`] (migrated to [gvenzl/oracle-free](#))
- [Sqitch](#) [`8b38027`]
- [Testcontainers](#) [`99b91b8`]
- [Upscheme](#) [`954650a`]
- [utPLSQL](#) [`327110f`]
 - [utPLSQL-maven-plugin](#) [`4d8deeb`]
 - [utPLSQL-java-api](#) [`a4a0eb5`]
 - [utPLSQL-demo-project](#) [`99463f0`]
- [XWiki](#) [`e677893`]

If you are using these images and would like to be listed as well, please open an [issue on GitHub](#) or reach out on [Twitter](#).

How to use this image

Subtle differences between versions

The 11gR2 (11.2.0.2) Oracle Database version stores the database data files under `/u01/app/oracle/oradata/XE`.

A volume for 11gR2 has to be pointed at `/u01/app/oracle/oradata` !

Environment variables

Environment variables allow you to customize your container. Note that these variables will only be considered during the database initialization (first container startup).

ORACLE_PASSWORD

This variable is mandatory for the first container startup and specifies the password for the Oracle Database `SYS` and `SYSTEM` users.

ORACLE_RANDOM_PASSWORD

This is an optional variable. Set this variable to a non-empty value, like `yes`, to generate a random initial password for the `SYS` and `SYSTEM` users. The generated password will be printed to stdout (`ORACLE_PASSWORD_FOR_SYS_AND_SYSTEM: ...`).

ORACLE_DATABASE (for 18c and onwards)

This is an optional variable. Set this variable to a non-empty string to create a new pluggable database with the name specified in this variable.

Note: this variable is only supported for Oracle Database XE 18c and onwards; 11g does not support pluggable databases.

Note: creating a new database will add to the initial container startup time. If you do not want that additional startup time, use the already existing `XEPDB1` database instead.

APP_USER

This is an optional variable. Set this variable to a non-empty string to create a new database schema user with the name specified in this variable. For 18c and onwards, the user will be created in the default `XEPDB1` pluggable database. If `ORACLE_DATABASE` has been specified, the user will also be created in that pluggable database. This variable requires `APP_USER_PASSWORD` or `APP_USER_PASSWORD_FILE` to be specified as well.

APP_USER_PASSWORD

This is an optional variable. Set this variable to a non-empty string to define a password for the database schema user specified by `APP_USER`. This variable requires `APP_USER` to be specified as well.

GitHub Actions

The images can be used as a [Service Container](#) within a [GitHub Actions](#) workflow. Below is an example service definition for your GitHub Actions YAML file:

```
services:

  # Oracle service (label used to access the service container)
  oracle:

    # Docker Hub image (feel free to change the tag "latest" to
    any other available one)
    image: gvenzl/oracle-xe:latest

    # Provide passwords and other environment variables to conta
    iner
    env:
      ORACLE_RANDOM_PASSWORD: true
      APP_USER: my_user
      APP_USER_PASSWORD: my_password_which_I_really_should_chang
    e

    # Forward Oracle port
    ports:
      - 1521:1521

    # Provide healthcheck script options for startup
    options: >-
      --health-cmd healthcheck.sh
      --health-interval 10s
      --health-timeout 5s
      --health-retries 10
```

After your service is created, you can connect to it via the following properties:

- Hostname:
 - `oracle` (from within another container)

- `localhost` or `127.0.0.1` (from the host directly)
- Port: `1521`
- Service name: `FREEPDB1`
- Database App User: `my_user`
- Database App Password: `my_password_which_I_really_should_change`

If you amend the variables above, here is some more useful info:

- Ports: you can access the port dynamically via `${{ job.services.oracle.ports[1521] }}`. This is helpful when you do not want to specify a given port via `- 1521/tcp` instead of `- 1521:1521`. Note that the `oracle` refers to the service name in the yaml file. If you call your service differently, you will also have to change `oracle` here to that other service name.
- Database Admin User: `system`
- Database Admin User Password: `$ORACLE_PASSWORD`
- Database App User: `$APP_USER`
- Database App User Password: `$APP_USER_PASSWORD`
- Example JDBC connect string with dynamic port allocation:
`jdbc:oracle:thin:@localhost:${{ job.services.oracle.ports[1521] }}/XEPDB1`

Image flavors

Flavor	Extension	Description	Use cases
Slim	<code>-slim</code>	An image focussed on smallest possible image size instead of additional functionality.	Wherever small images sizes are important but advanced functionality of Oracle Database is not needed.

Flavor	Extension	Description	Use cases
Regular	[None]	A well-balanced image between image size and functionality. Recommended for most use cases.	Recommended for most use cases.
Full	<code>-full</code>	An image containing all functionality as provided by the Oracle Database installation.	Best for extensions and/or customizations.
Faststart	<code>*-faststart</code>	The same image flavor as above but with an already expanded and ready to go database inside the image. This image trades image size on disk for a faster database startup time.	Best for (automated) test scenarios where the image is pulled once and many containers started and torn down with no need of persistency (container volumes).

For a full list of changes that have been made to the Oracle Database and OS installation in each individual image flavor, please see [ImageDetails.md](#) [↗](#).

Database users

The image provides a built-in command `createAppUser` to create additional Oracle Database users with standard privileges. The same command is also executed when the `APP_USER` environment variable is specified. If you need just one additional database user for your application, the `APP_USER` environment variable is the best approach. However, if you need multiple users, you can execute the command for each individual user directly:

Usage:

```
createAppUser APP_USER APP_USER_PASSWORD [TARGET_PDB]
```

APP_USER: the user name of the new user

APP_USER_PASSWORD: the password for that user

TARGET_PDB: the target pluggable database the user should be created in, default XEPDB1 (ignored for 11g R2)

Example:

```
docker exec <container name|id> createAppUser <your app user> <your app user password> [<your target PDB>]
```

The command can also be invoked inside initialization and/or startup scripts.

Container secrets

As an alternative to passing sensitive information via environment variables, `_FILE` may be appended to some of the previously listed environment variables, causing the initialization script to load the values for those variables from files present in the container. In particular, this can be used to load passwords from Container/Docker secrets stored in `/run/secrets/<secret_name>` files. For example:

```
docker run --name some-oracle -e ORACLE_PASSWORD_FILE=/run/secrets/oracle-passwd -d gvenzl/oracle-xe
```

This mechanism is supported for:

- `APP_USER_PASSWORD`
- `ORACLE_PASSWORD`
- `ORACLE_DATABASE`

Note: there is a significant difference in how containerization technologies handle secrets. For more information on that topic, please consult the official containerization technology documentation:

- [Docker](#) ↗
- [Podman](#) ↗
- [Kubernetes](#) ↗

Initialization scripts

If you would like to perform additional initialization of the database running in a container, you can add one or more `*.sql` , `*.sql.gz` , `*.sql.zip` or `*.sh` files under `/container-entrypoint-initdb.d` (creating the directory if necessary). After the database setup is completed, these files will be executed automatically in alphabetical order.

The directory can include sub directories which will be traversed recursively in alphabetical order alongside the files. The container does not give any priority to files or directories, meaning that whatever comes next in alphabetical order will be processed next. If it is a file it will be executed, if it is a directory it will be traversed. To guarantee the order of execution, consider using a clear prefix in your file and directory names like numbers `001_` , `002_` . This will also make it easier for any user to understand which script is supposed to be executed in what order.

The `*.sql` , `*.sql.gz` and `*.sql.zip` files ***will be executed in SQL*Plus as the SYS user connected to the Oracle instance (XE)***. This allows users to modify instance parameters, create new pluggable databases, tablespaces, users and more as part of their initialization scripts. ***If you want to initialize your application schema, you first have to connect to that schema inside your initialization script!*** Compressed files will be uncompressed on the fly, allowing for e.g. bigger data loading scripts to save space.

Executable `*.sh` files will be run in a new shell process while non-executable `*.sh` files (files that do not have the Linux `x` executable permission set) will be sourced into the current shell process. The main difference between these methods is that sourced shell scripts can influence the environment of the current process and should generally be avoided. However, sourcing scripts allows for execution of these scripts even if the executable flag is not set for the files containing them. This basically avoids the "why did my script not get executed" confusion.

Note: scripts in `/container-entrypoint-initdb.d` are only run the first time the database is initialized; any pre-existing database will be left untouched on container startup.

Note: you can also put files under the `/docker-entrypoint-initdb.d` directory. This is kept for backwards compatibility with other widely used container images but should generally be avoided. Do not put files under `/container-entrypoint-initdb.d` and `/docker-entrypoint-initdb.d` as this would cause the same files to be executed twice!

Warning: if a command within the sourced `/container-entrypoint-initdb.d` scripts fails, it will cause the main entrypoint script to exit and stop the container. It also may leave the database in an incomplete initialized state. Make sure that shell scripts handle error situations gracefully and ideally do not source them!

Warning: do not exit executable `/container-entrypoint-initdb.d` scripts with a non-zero value (using e.g. `exit 1;`) unless it is desired for a container to be stopped! A non-zero return value will tell the main entrypoint script that something has gone wrong and that the container should be stopped.

Example

The following example installs the [countries, cities and currencies sample data set](#)  under a new user `TEST` into the database:

```
[gvenzl@localhost init_scripts]$ pwd
/home/gvenzl/init_scripts

[gvenzl@localhost init_scripts]$ ls -al
total 12
drwxrwxr-x  2 gvenzl gvenzl  61 Mar  7 11:51 .
drwx----- 19 gvenzl gvenzl 4096 Mar  7 11:51 ..
-rw-rw-r--  1 gvenzl gvenzl  134 Mar  7 11:50 1_create_user.sql
-rwxrwxr-x  1 gvenzl gvenzl  164 Mar  7 11:51 2_create_data_model.s
h

[gvenzl@localhost init_scripts]$ cat 1_create_user.sql
ALTER SESSION SET CONTAINER=XEPDB1;

CREATE USER TEST IDENTIFIED BY test QUOTA UNLIMITED ON USERS;

GRANT CONNECT, RESOURCE TO TEST;
```

```
[gvenzl@localhost init_scripts]$ cat 2_create_data_model.sh
curl -LJO https://raw.githubusercontent.com/gvenzl/sample-data/master/countries-cities-currencies/install.sql

sqlplus -s test/test@//localhost/XEPDB1 @install.sql

rm install.sql
```

As the execution happens in alphabetical order, numbering the files will guarantee the execution order. A new container started up with `/home/gvenzl/init_scripts` pointing to `/container-entrypoint-initdb.d` will then execute the files above:

```
docker run --name test \
> -p 1521:1521 \
> -e ORACLE_RANDOM_PASSWORD="y" \
> -v /home/gvenzl/init_scripts:/container-entrypoint-initdb.d \
> gvenzl/oracle-xe:18.4.0-full
CONTAINER: starting up...
CONTAINER: first database startup, initializing...
...
CONTAINER: Executing user defined scripts...
CONTAINER: running /container-entrypoint-initdb.d/1_create_user.sql
...

Session altered.

User created.

Grant succeeded.

CONTAINER: DONE: running /container-entrypoint-initdb.d/1_create_user.sql

CONTAINER: running /container-entrypoint-initdb.d/2_create_data_model
```

```

l.sh ...
% Total      % Received % Xferd  Average Speed   Time    Time       Ti
me  Current                                  Dload  Upload   Total   Spent    Le
ft  Speed
100 115k  100  115k    0     0   460k      0 --:--:-- --:--:-- --:--
-:-- 460k

```

Table created.

```

...
Table                provided actual
-----
regions                7         7
countries              196        196
cities                 204        204
currencies             146        146
currencies_countries   203        203

```

Thank you!

The installation is finished, please check the verification output above!

If the 'provided' and 'actual' row counts match, the installation was successful

.

If the row counts do not match, please check the above output for error messages

.

CONTAINER: DONE: running /container-entrypoint-initdb.d/2_create_data_model.sh

CONTAINER: DONE: Executing user defined scripts.

```
#####  
DATABASE IS READY TO USE!  
#####  
...
```

As a result, one can then connect to the new schema directly:

```
[gvenzl@localhost init_scripts]$ sqlplus test/test@//localhost/XEPDB  
1  
  
SQLcl: Release 20.3 Production on Sun Mar 07 12:05:06 2021  
  
Copyright (c) 1982, 2021, Oracle. All rights reserved.  
  
Connected to:  
Oracle Database 18c Express Edition Release 18.0.0.0.0 - Production  
Version 18.4.0.0.0  
  
SQL> select * from countries where name = 'Austria';  
  
COUNTRY_ID COUNTRY_CODE NAME OFFICIAL_NAME POPULATION AREA_  
SQ_KM LATITUDE LONGITUDE TIMEZONE REGION_ID  
-----  
-----  
AUT AT Austria Republic of Austria 8793000  
83871 47.33333 13.33333 Europe/Vienna EU  
  
SQL>
```

Startup scripts

If you would like to perform additional action after the database running in a container has been started, you can add one or more `*.sql`, `*.sql.gz`, `*.sql.zip` or `*.sh` files under `/container-entrypoint-startdb.d` (creating the directory if necessary). After the database is up and ready for requests, these files will be executed automatically in alphabetical order.

The execution order and implications are the same as with the [Initialization scripts](#) described above.

Note: you can also put files under the `/docker-entrypoint-startdb.d` directory. This is kept for backwards compatibility with other widely used container images but should generally be avoided. Do not put files under `/container-entrypoint-startdb.d` and `/docker-entrypoint-startdb.d` as this would cause the same files to be executed twice!

Note: if the database inside the container is initialized (started for the first time), startup scripts are executed after the setup scripts.

Warning: files placed in `/container-entrypoint-startdb.d` are always executed after the database in a container is started, including pre-created databases. Use this mechanism only if you wish to perform a certain task always after the database has been (re)started by the container.

Feedback

If you have questions or constructive feedback about these images, please file a ticket over at github.com/gvenzl/oci-oracle-xe ↗.



Why

Overview
What is a Container

Products

Product Overview

Product Offerings

Docker Desktop
Docker Hub

Features

Container Runtime
Developer Tools
Docker App
Kubernetes

Developers

Getting Started
Play with Docker
Community
Open Source
Documentation

Company

About Us
Resources
Blog
Customers
Partners
Newsroom
Events and Webinars
Careers
Contact Us
System Status ↗

